GroupDNA — WhatsApp Group Analytics

Student: B HARSHITHA
Roll Number:
Batch: Data Analytics September 2026 Batch

Internship: Data Analytics Internship — The Unlox Academy
Project: Week 1 Industry-Graded Minor Project
Date: 21 September 2026

> **Objective:** Decode the supplied `hostel_bois.txt` WhatsApp export into a clean, screenshot-ready behavioural analytics report using Python fundamentals and NumPy.

Project constraints and approach

In this it shows the brief project constraints:

Allowed: Python basics, lists, tuples, sets, dictionaries, loops, functions, NumPy, file handling, and datetime.

Not allowed: pandas, matplotlib, seaborn, Counter, regex, and machine-learning libraries.

Feature 1 — Chat Parser

Reads the WhatsApp export, handles system/media/deleted messages, and validates the expected counts.

In [11…

```
from datetime import datetime, timedelta
import numpy as np

FILE_NAME = "hostel_bois.txt"
EXPECTED_DAYS = 60

PEOPLE_ORDER = ["Rahul", "Priya", "Aman", "Karan", "Neha", "Vikas"]

STOP_WORDS = set("""
i me my we our you your he him his she her it its they them their
a an the and or but if because as of at by for with about into through
```

```
during before after above below to from up down in out on off over under
again further then once here there when where why how all any both each
few more most other some such no nor not only own same so than too very
can will just should now am is are was were be been being have has had
having do does did doing would could might must what which who whom whose
this that these those
""".lower().split())

PUNCTUATION = ".,!?;:'\"()[]{}<>/\\|@#$%^&*_+=-`~"

CARING_KEYWORDS = [
    "okay", "safe", "eat", "sleep", "take care",
    "are you", "please", "reminder", "drink water", "don't forget"
]

COMEDY_KEYWORDS = ["lol", "lmao", "haha", "rofl", "lmfao"]
def looks_like_date_prefix(line):
    """Check DD/MM/YY without using regex."""
    return (
        len(line) >= 8
        and line[2] == "/"
        and line[5] == "/"
        and line[0:2].isdigit()
        and line[3:5].isdigit()
        and line[6:8].isdigit()
    )

def parse_chat_file(file_name):
    """
    Parse a WhatsApp text export.
    User messages are retained, including media/deleted entries with a st
    System messages are skipped from the message list.
    Continuation lines are appended to the previous message.
    """
    parsed = []
    system_count = 0

    try:
        file_handle = open(file_name, "r", encoding="utf-8")
    except FileNotFoundError:
        file_handle = open("/content/" + file_name, "r", encoding="utf-8"

    with file_handle as f:
        raw_lines = f.readlines()

    for raw in raw_lines:
        line = raw.strip()

        if line == "":
            continue

        if not looks_like_date_prefix(line):
            if parsed:
                parsed[-1]["text"] += " " + line
```

```
            continue

        if " - " not in line:
            system_count += 1
            continue

        timestamp, remainder = line.split(" - ", 1)

        if ": " not in remainder:
            system_count += 1
            continue

        sender, text = remainder.split(": ", 1)

        try:
            dt = datetime.strptime(timestamp, "%d/%m/%y, %H:%M")
        except ValueError:
            system_count += 1
            continue

        status = "normal"
        if text == "<Media omitted>":
            status = "media"
        elif text == "This message was deleted":
            status = "deleted"

        parsed.append({
            "timestamp": timestamp,
            "datetime": dt,
            "sender": sender,
            "text": text,
            "status": status
        })

    return parsed, system_count

messages, system_messages = parse_chat_file(FILE_NAME)

media_messages = [m for m in messages if m["status"] == "media"]
deleted_messages = [m for m in messages if m["status"] == "deleted"]
normal_messages = [m for m in messages if m["status"] == "normal"]

participants = sorted(set(m["sender"] for m in messages))

print("=" * 72)
print("GROUPDNA | PARSER CHECK")
print("=" * 72)
print(f"User messages retained : {len(messages):,}")
print(f"Normal text messages   : {len(normal_messages):,}")
print(f"Media omitted          : {len(media_messages):,}")
print(f"Deleted messages       : {len(deleted_messages):,}")
print(f"System messages        : {system_messages}")
print(f"Participants            : {len(participants)}")
print(f"Date range              : {min(m['datetime'] for m in messages):%
```

```
assert len(messages) == 3174, "Parser check failed: expected 3,174 user m
assert len(media_messages) == 32, "Media count check failed."
assert len(deleted_messages) == 15, "Deleted count check failed."
assert system_messages == 4, "System-message count check failed."
assert len(participants) == 6, "Participant count check failed."

print("\nParser validation: PASSED")
```

```
========================================================================
GROUPDNA | PARSER CHECK
========================================================================
User messages retained : 3,174
Normal text messages   : 3,127
Media omitted          : 32
Deleted messages       : 15
System messages        : 4
Participants            : 6
Date range              : 01 April 2024 to 30 May 2024

Parser validation: PASSED
```

Feature 2 — Group Overview

Calculates the overall message period, participants and per-person message share.

In [12…

```
def message_counts_by_person(data):
    counts = {}
    for message in data:
        person = message["sender"]
        counts[person] = counts.get(person, 0) + 1
    return counts

message_counts = message_counts_by_person(messages)
first_date = min(m["datetime"].date() for m in messages)
last_date = max(m["datetime"].date() for m in messages)
period_days = (last_date - first_date).days + 1

print("\n" + "=" * 72)
print("GROUP OVERVIEW")
print("=" * 72)
print("Group       : Hostel Bois 4ever")
print(f"Period      : {first_date:%d %B %Y} to {last_date:%d %B %Y} ({per
print(f"Total msgs  : {len(messages):,}")
print(f"Participants: {len(participants)}")
print("\nMESSAGES PER PERSON")
print("-" * 72)

ranked_counts = sorted(message_counts.items(), key=lambda item: item[1],
for person, count in ranked_counts:
    percentage = count / len(messages) * 100
    print(f"{person:<10} : {count:>4} ({percentage:5.1f}%)")
```

```
assert sum(message_counts.values()) == 3174
```

```
========================================================================
GROUP OVERVIEW
========================================================================
Group       : Hostel Bois 4ever
Period      : 01 April 2024 to 30 May 2024 (60 days)
Total msgs  : 3,174
Participants: 6

MESSAGES PER PERSON
------------------------------------------------------------------------
Rahul      :  953 ( 30.0%)
Priya      :  718 ( 22.6%)
Neha       :  635 ( 20.0%)
Aman       :  490 ( 15.4%)
Karan      :  354 ( 11.2%)
Vikas      :   24 (  0.8%)
```

Feature 3 — Most Active Day and Hour

Finds the busiest calendar day and the highest-volume hour.

In [13…

```
def count_by_date(data):
    counts = {}
    for message in data:
        key = message["datetime"].date()
        counts[key] = counts.get(key, 0) + 1
    return counts

def count_by_hour(data):
    counts = {}
    for message in data:
        key = message["datetime"].hour
        counts[key] = counts.get(key, 0) + 1
    return counts

daily_counts = count_by_date(messages)
hourly_counts = count_by_hour(messages)

busiest_day, busiest_day_count = max(daily_counts.items(), key=lambda ite
busiest_hour, busiest_hour_count = max(hourly_counts.items(), key=lambda

print("\n" + "=" * 72)
print("MOST ACTIVE DAY & HOUR")
print("=" * 72)
print(f"Busiest day  : {busiest_day:%d %B %Y} ({busiest_day_count} messag
print(f"Busiest hour : {busiest_hour:02d}:00 - {(busiest_hour + 1) % 24:0
```

```
========================================================================
MOST ACTIVE DAY & HOUR
========================================================================
Busiest day  : 04 May 2024 (76 messages)
Busiest hour : 18:00 - 19:00 (248 messages)
```

Feature 4 — NumPy Activity Heatmap

Builds a 6 × 24 NumPy matrix and renders it with terminal-style shading.

In [14…

```python
person_index = {}
for index, person in enumerate(PEOPLE_ORDER):
    person_index[person] = index

activity_matrix = np.zeros((len(PEOPLE_ORDER), 24), dtype=int)

for message in messages:
    row = person_index[message["sender"]]
    hour = message["datetime"].hour
    activity_matrix[row, hour] += 1

print("\n" + "=" * 72)
print("ACTIVITY HEATMAP | MESSAGES BY HOUR")
print("=" * 72)
print("       " + "".join(f"{h:02d} " for h in range(24)))
print("       " + "-" * 72)

for row_index, person in enumerate(PEOPLE_ORDER):
    row = activity_matrix[row_index]
    row_max = np.max(row)
    symbols = []

    for value in row:
        ratio = value / row_max if row_max else 0
        if value == 0:
            symbol = "."
        elif ratio <= 0.25:
            symbol = "░"
        elif ratio <= 0.50:
            symbol = "▒"
        else:
            symbol = "█"
        symbols.append(symbol)

    print(f"{person:<7}" + "  ".join(symbols))

print("\nHeatmap row totals:")
for index, person in enumerate(PEOPLE_ORDER):
    print(f"{person:<10}: {int(np.sum(activity_matrix[index])):>4} messag

assert int(np.sum(activity_matrix)) == len(messages)
```

```
========================================================================
ACTIVITY HEATMAP | MESSAGES BY HOUR
========================================================================
       00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21
22 23

------------------------------------------------------------------------
Rahul  ▒  ▒  ▒  ▒  ▒  ▒  ▒  ▒  ▒  ▒  ▒  ▒  ▉  ▓  ▓  ▉  ▉  ▓  ▉  ▉  ▓  ▉
▉  ▉
Priya  .  .  .  .  .  .  ▒  ▓  ▉  ▉  ▉  ▉  ▉  ▉  ▉  ▓  ▓  ▉  ▉  ▉  ▉  ▓
▓  ▒
Aman   ▉  ▉  ▉  ▉  ▉  .  .  .  .  .  .  .  .  .  .  ▒  ▒  ▒  ▒  ▒  ▒  ▒  ▒
.  ▉
Karan  .  .  .  .  .  .  .  ▒  ▓  ▓  ▉  ▓  ▉  ▉  ▉  ▉  ▉  ▉  ▉  ▉  ▉  ▓
▒  ▒
Neha   .  .  .  .  .  ▓  ▒  ▒  ▉  ▉  ▉  ▓  ▉  ▉  ▓  ▒  ▉  ▉  ▉  ▉  ▉  ▓
▓  ▓
Vikas  .  .  .  .  .  .  .  ▓  ▉  ▓  ▓  .  ▉  ▉  .  ▓  ▓  ▉  ▉  ▉  ▓  ▓
▓  ▉

Heatmap row totals:
Rahul     :  953 messages
Priya     :  718 messages
Aman      :  490 messages
Karan     :  354 messages
Neha      :  635 messages
Vikas     :   24 messages
```

Feature 5 — Top Words

Normalizes text, removes punctuation and selected stop words, then builds a custom frequency dictionary.

In [15…

```python
def clean_word(word):
    return word.lower().strip(PUNCTUATION)

word_counts = {}

for message in normal_messages:
    for raw_word in message["text"].split():
        word = clean_word(raw_word)

        if word == "" or word in STOP_WORDS or word.isdigit():
            continue

        word_counts[word] = word_counts.get(word, 0) + 1

top_words = sorted(
    word_counts.items(),
    key=lambda item: (-item[1], item[0])
)[:10]

print("\n" + "=" * 72)
```

```
print("THIS GROUP'S FAVOURITE WORDS")
print("=" * 72)

largest_word_count = top_words[0][1]
for word, count in top_words:
    bar_length = max(1, int(count / largest_word_count * 30))
    print(f"{word:<15} {'' * bar_length} {count}")
```

```
========================================================================
THIS GROUP'S FAVOURITE WORDS
========================================================================
guys            ############################## 318
hai             ######################### 268
today           ######################## 257
everyone        ################## 187
telling         ################# 179
bhai            ################ 160
one             ############### 157
started         ############### 150
entire          ############## 145
scene           ############## 145
```

Feature 6 — Response Speed and Silent Streaks

Uses datetime arithmetic for response gaps and checks every day in the 60-day period for silent streaks.

In [6]:
```
def average_response_gaps(data):
    gaps = {}

    for previous, current in zip(data, data[1:]):
        if current["sender"] == previous["sender"]:
            continue

        seconds = (current["datetime"] - previous["datetime"]).total_seco
        person = current["sender"]

        if person not in gaps:
            gaps[person] = []

        gaps[person].append(seconds)

    averages = {}
    for person in PEOPLE_ORDER:
        if person in gaps and gaps[person]:
            averages[person] = sum(gaps[person]) / len(gaps[person])
        else:
            averages[person] = 0

    return averages, gaps

def longest_silent_streak(active_day_set, all_days):
    longest = 0
```

```
    current = 0

    for day in all_days:
        if day not in active_day_set:
            current += 1
            if current > longest:
                longest = current
        else:
            current = 0

    return longest

response_averages, response_gap_lists = average_response_gaps(messages)

all_days = []
for offset in range(period_days):
    all_days.append(first_date + timedelta(days=offset))

active_days = {}
silent_streaks = {}

for person in PEOPLE_ORDER:
    active_days[person] = set(
        m["datetime"].date()
        for m in messages
        if m["sender"] == person
    )
    silent_streaks[person] = longest_silent_streak(active_days[person], a

def format_duration(seconds):
    minutes = seconds / 60
    if minutes < 60:
        return f"{minutes:.1f} minutes"
    return f"{minutes / 60:.1f} hours"

fastest = min(response_averages.items(), key=lambda item: item[1])
slowest = max(response_averages.items(), key=lambda item: item[1])

print("\n" + "=" * 72)
print("RESPONSE PATTERNS")
print("=" * 72)
print(f"Fastest average response : {fastest[0]} ({format_duration(fastest
print(f"Slowest average response : {slowest[0]} ({format_duration(slowest

print("\nLONGEST SILENT STREAKS")
print("-" * 72)
for person, streak in sorted(silent_streaks.items(), key=lambda item: ite
    print(f"{person:<10}: {streak} days")
```

```
================================================================================
RESPONSE PATTERNS
================================================================================
Fastest average response : Rahul (34.9 minutes)
Slowest average response : Aman (55.4 minutes)

LONGEST SILENT STREAKS
--------------------------------------------------------------------------------
Vikas     : 11 days
Rahul     : 0 days
Priya     : 0 days
Aman      : 0 days
Karan     : 0 days
Neha      : 0 days
```

Feature 7 — Personality Archetype Detection

Uses explicit quantitative rules and an exclusive highest-score assignment.

In [16…

```python
def average_burst_length(data):
    bursts = {person: [] for person in PEOPLE_ORDER}
    previous_sender = None
    current_burst = 0

    for message in data:
        sender = message["sender"]

        if sender == previous_sender:
            current_burst += 1
        else:
            if previous_sender is not None:
                bursts[previous_sender].append(current_burst)
            previous_sender = sender
            current_burst = 1

    if previous_sender is not None:
        bursts[previous_sender].append(current_burst)

    averages = {}
    for person in PEOPLE_ORDER:
        if bursts[person]:
            averages[person] = sum(bursts[person]) / len(bursts[person])
        else:
            averages[person] = 0

    return averages

burst_averages = average_burst_length(messages)

def count_caring_keywords(person):
    total = 0
    for message in normal_messages:
        if message["sender"] != person:
```

```
            continue
        lower_text = message["text"].lower()
        for keyword in CARING_KEYWORDS:
            total += lower_text.count(keyword)
    return total

def night_message_ratio(person):
    person_messages = [m for m in messages if m["sender"] == person]
    if not person_messages:
        return 0
    night_count = 0
    for message in person_messages:
        hour = message["datetime"].hour
        if hour >= 23 or hour <= 4:
            night_count += 1
    return night_count / len(person_messages)

def average_words(person):
    person_messages = [
        m for m in normal_messages if m["sender"] == person
    ]
    if not person_messages:
        return 0
    return sum(len(m["text"].split()) for m in person_messages) / len(per

def drama_ratio(person):
    person_messages = [
        m for m in normal_messages if m["sender"] == person
    ]
    if not person_messages:
        return 0

    dramatic = 0
    for message in person_messages:
        text = message["text"]
        if (len(text) >= 3 and text.isupper()) or text.count("!") >= 2:
            dramatic += 1

    return dramatic / len(person_messages)

def comedy_ratio(person):
    person_messages = [
        m for m in normal_messages if m["sender"] == person
    ]
    if not person_messages:
        return 0

    funny_hits = 0
    for message in person_messages:
        lower_text = message["text"].lower()
        for keyword in COMEDY_KEYWORDS:
            funny_hits += lower_text.count(keyword)

    return funny_hits / len(person_messages)
```

```
def question_ratio(person):
    person_messages = [
        m for m in normal_messages if m["sender"] == person
    ]
    if not person_messages:
        return 0

    questions = 0
    for message in person_messages:
        if message["text"].rstrip().endswith("?"):
            questions += 1

    return questions / len(person_messages)

archetype_scores = {}

max_caring = max(count_caring_keywords(p) for p in PEOPLE_ORDER)

for person in PEOPLE_ORDER:
    spam_score = (burst_averages[person] - 3.0) / 3.0 if burst_averages[p
    mom_score = count_caring_keywords(person) / max_caring if count_carin
    night_ratio = night_message_ratio(person)
    night_score = night_ratio if night_ratio > 0.60 else 0
    story_words = average_words(person)
    story_score = story_words / 30.0 if story_words > 30 else 0
    drama_ratio_value = drama_ratio(person)
    drama_score = drama_ratio_value if drama_ratio_value > 0.30 else 0
    ghost_ratio = 1 - len(active_days[person]) / len(all_days)
    ghost_score = ghost_ratio if ghost_ratio > 0.60 else 0
    comedian_ratio = comedy_ratio(person)
    question_ratio_value = question_ratio(person)

    archetype_scores[person] = {
        "THE SPAMMER": spam_score,
        "THE GROUP MOM": mom_score,
        "THE NIGHT OWL": night_score,
        "THE STORYTELLER": story_score,
        "THE DRAMA QUEEN": drama_score,
        "THE GHOST": ghost_score,
        "THE COMEDIAN": comedian_ratio if comedian_ratio > 0 else 0,
        "THE QUESTION MASTER": question_ratio_value if question_ratio_val
    }
assigned_archetypes = {}
runner_ups = {}

for person in PEOPLE_ORDER:
    ranked = sorted(
        archetype_scores[person].items(),
        key=lambda item: item[1],
        reverse=True
    )
    assigned_archetypes[person] = ranked[0][0]
    runner_ups[person] = ranked[1][0]
```

```
print("\n" + "=" * 72)
print("PERSONALITY ARCHETYPES")
print("=" * 72)

for person in PEOPLE_ORDER:
    label = assigned_archetypes[person]
    score = archetype_scores[person][label]
    print(f"{person:<10} → {label:<20} score={score:.3f} | runner-up: {ru
```

```
========================================================================
PERSONALITY ARCHETYPES
========================================================================
Rahul      → THE SPAMMER          score=0.506 | runner-up: THE COMEDIAN
Priya      → THE GROUP MOM        score=1.000 | runner-up: THE QUESTION M
ASTER
Aman       → THE NIGHT OWL        score=0.798 | runner-up: THE SPAMMER
Karan      → THE STORYTELLER      score=1.902 | runner-up: THE SPAMMER
Neha       → THE DRAMA QUEEN      score=0.633 | runner-up: THE SPAMMER
Vikas      → THE GHOST            score=0.733 | runner-up: THE COMEDIAN
```

Bonus — Ninth Archetype

Adds a separately reported, Indian-college-context archetype without changing the mandatory eight-rule assignment.

In [17…

```
CAMPUS_WORDS = [
    "class", "professor", "college", "placement", "hostel",
    "campus", "assignment", "exam", "canteen", "library"
]

def campus_connector_score(person):
    hits = 0
    person_messages = [m for m in normal_messages if m["sender"] == perso
    for message in person_messages:
        lower_text = message["text"].lower()
        for word in CAMPUS_WORDS:
            hits += lower_text.count(word)
    return hits

campus_scores = {
    person: campus_connector_score(person)
    for person in PEOPLE_ORDER
}
campus_leader = max(campus_scores.items(), key=lambda item: item[1])

print("\nBONUS ARCHETYPE")
print(f"{campus_leader[0]} → THE CAMPUS CONNECTOR ({campus_leader[1]} cam
```

```
BONUS ARCHETYPE
Karan → THE CAMPUS CONNECTOR (373 campus-related keyword hits)
```

Feature 8 — Final Report

Produces the polished, screenshot-ready analytics summary.

In [23…

```
print("\n" + "=" * 72)
print("GROUPDNA | FINAL ANALYTICS REPORT")
print("=" * 72)
print("Group        : Hostel Bois 4ever")
print("Analyst      : B HARSSHITHA")
print("Project      : Unlox Academy | Week 1 Minor Project")
print(f"Period       : {first_date:%d %B %Y} to {last_date:%d %B %Y}")
print(f"Messages     : {len(messages):,}")
print(f"Participants : {len(participants)}")
print("-" * 72)

top_person = ranked_counts[0]
print(f"Most active  : {top_person[0]} ({top_person[1]:,} messages)")
print(f"Busiest day  : {busiest_day:%d %B %Y} ({busiest_day_count} messag
print(f"Peak hour    : {busiest_hour:02d}:00 - {(busiest_hour + 1) % 24:0
print(f"Fastest reply: {fastest[0]} ({format_duration(fastest[1])})")
print(f"Longest ghost: {max(silent_streaks.items(), key=lambda item: item

print("\nARCHETYPE SNAPSHOT")
for person in PEOPLE_ORDER:
    print(f"  {person:<10} {assigned_archetypes[person]}")

print("\nDATA QUALITY")
print(f"  System messages skipped : {system_messages}")
print(f"  Media entries tracked   : {len(media_messages)}")
print(f"  Deleted entries tracked : {len(deleted_messages)}")
print("=" * 72)

print("\nPROJECT VALIDATION")
print("✓ Parser checks passed")
print("✓ NumPy heatmap total matches 3,174 messages")
print("✓ Six participants detected")
print("✓ All mandatory analytics sections executed")
print("✓ No pandas, matplotlib, seaborn, plotly, regex, Counter, or ML li
```

```
================================================================================
GROUPDNA | FINAL ANALYTICS REPORT
================================================================================
Group        : Hostel Bois 4ever
Analyst      : B HARSSHITHA
Project      : Unlox Academy | Week 1 Minor Project
Period       : 01 April 2024 to 30 May 2024
Messages     : 3,174
Participants : 6
--------------------------------------------------------------------------------
Most active  : Rahul (953 messages)
Busiest day  : 04 May 2024 (76 messages)
Peak hour    : 18:00 - 19:00
Fastest reply: Rahul (34.9 minutes)
Longest ghost: Vikas (11 days)

ARCHETYPE SNAPSHOT
  Rahul      THE SPAMMER
  Priya      THE GROUP MOM
  Aman       THE NIGHT OWL
  Karan      THE STORYTELLER
  Neha       THE DRAMA QUEEN
  Vikas      THE GHOST

DATA QUALITY
  System messages skipped : 4
  Media entries tracked   : 32
  Deleted entries tracked : 15
================================================================================

PROJECT VALIDATION
✓ Parser checks passed
✓ NumPy heatmap total matches 3,174 messages
✓ Six participants detected
✓ All mandatory analytics sections executed
✓ No pandas, matplotlib, seaborn, plotly, regex, Counter, or ML libraries
used
```

Reflection

This project helped me understand how raw text data can be converted into structured analytics without relying on high-level data-analysis libraries.

I learned how to read and process a WhatsApp chat text file using Python.

I learned to use lists, dictionaries, loops, and functions.

I learned how to analyze messages and calculate activity patterns.

I learned how to use NumPy for the activity heatmap.

This project improved my understanding of basic Python and data analytics.